

Part 7

```
PS C:\Users\rkory\OneDrive\Desktop\FSC8561-Labs-Ryan\Lab 1> python server.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 59860)
Received: HELLO|Ryan
Username: Ryan
Received: MSG|hello
Ryan says: hello
Received: MSG|goodbye
Ryan says: goodbye
Received: EXIT|
Server closed
```

```
PS C:\Users\rkory\OneDrive\Desktop\FSC8561-Labs-Ryan\Lab 1> python client.py
Enter your username: Ryan
Server: OK|Hello Ryan
Enter message or type EXIT to leave: hello
Server: OK|Message received from Ryan
Enter message or type EXIT to leave: goodbye
Server: OK|Message received from Ryan
Enter message or type EXIT to leave: exit
Server: OK|Goodbye
Disconnected
```

The server now stays on until the client exits the program.

Part 8

Empty Message

```
PS C:\Users\rkory\OneDrive\Desktop\FSC8561-Labs-Ryan\Lab 1> python client.py
Enter your username: Ryan
Server: OK|Hello Ryan
Enter message or type EXIT to leave:
Server: ERROR|Message cannot be empty
Enter message or type EXIT to leave:
```

Longer than 200 characters

```
PS C:\Users\rkory\OneDrive\Desktop\FSC8561-Labs-Ryan\Lab 1> python client.py
Enter your username: Ryan
Server: OK|Hello Ryan
Enter message or type EXIT to leave:
Server: ERROR|Message cannot be empty
Enter message or type EXIT to leave: this is a message longer than 200 charactersthis is a message longer than 200 chara
ctersthis is a message longer than 200 charactersthis is a message longer than 200 charactersthis is a message longer th
an 200 charactersthis is a message longer than 200 charactersthis is a message longer than 200 charactersthis is a messa
ge longer than 200 charactersthis is a message longer than 200 charactersthis is a message longer than 200 charactersthi
s is a message longer than 200 charactersthis is a message longer than 200 charactersthis is a message longer than 200 c
haractersthis is a message longer than 200 charactersthis is a message longer than 200 charactersthis is a message longe
r than 200 characters
Server: ERROR|Message too long
Enter message or type EXIT to leave:
```

Invalid Command Format

Modified code for testing

```
client_socket.send(  
    # hello_message.encode()  
    "INVALID MESSAGE".encode()  
)  
  
response = client_socket.recv(1024)
```

```
Windows PowerShell  
Copyright (C) Microsoft Corporation. All rights reserved.  
  
PS C:\Users\rkory> cd '..\OneDrive\Desktop\F SCT8561-Labs-Ryan\Lab 1\  
PS C:\Users\rkory\OneDrive\Desktop\F SCT8561-Labs-Ryan\Lab 1> python server.py  
Server is waiting for a connection...  
Connected by: ('127.0.0.1', 56522)  
Received: INVALID MESSAGE  
  
PS C:\Users\rkory\OneDrive\Desktop\F SCT8561-Labs-Ryan\Lab 1> python .\client.py  
Enter your username: Ryan  
Server: ERROR|Invalid command format  
Enter message or type EXIT to leave:
```

Part 10 Security Reflection

1.

If the server crashes while the client is connected, the tcp connection closes. The client may receive an empty result from `recv(1024)` or a “ConnectionResetError” or “BrokenPipeError” on a later operation. In our lab files, it will continue prompting the client or terminate with an exception.

2.

We could handle multiple clients by accepting connections in a loop and give each client its own thread, process, or asyncio task. A shared client registry could track connected users. Access to shared data would need synchronization, and each client should be isolated so one slow client does not block others

3.

Because TCP provides a reliable byte stream, and does not preserve message boundaries. Because the network can split data into packets of varying sizes, a single `send()` on the client side can result in multiple `recv()` calls on the server side. On the other hand, multiple `send()` calls can be batched into a single `recv()` call.

4.

TCP guarantees the ordered delivery and retransmission of bytes, but it does not provide encryption, authentication, authorization, protection from malicious input, or proof that the peer is trustworthy. Plain TCP traffic can be intercepted or modified.

5.

- Spoofed or duplicate usernames
- Oversized input
- Malformed commands
- Invalid encodings
- Command confusion
- Denial of service
- Log injection
- Future injection vulnerabilities if messages are stored, displayed or passed to another system

6.

A malicious client can

- repeatedly connect,
- send messages rapidly
- use very large usernames
- Send malformed bytes
- Send MSG before HELLO
- Impersonate another user
- Keep connections open indefinitely
- Send many invalid commands to consume server resources

7.

Replace unauthenticated HELLO|username with a login or handshake protocol

Ex:

- Use a registered username and password
- Issue a session token after successful login
- Authorize commands based on the authenticated identity
- Prevent duplicate sessions where appropriate
- Avoid sending passwords in plaintext

8.

Wrap the socket with TLS using python's SSL module. The server should use a certificate and private key, while clients verify the server certificate and hostname. TLS can also provide confidentiality, integrity and server authentication. Certificate validation should not be disabled in production.

9. The server currently validates

- Messages contain |

- HELLO has a non empty username
- MSG is sent only after HELLO
- Message content is non-empty
- Message content is a max of 200 characters
- EXIT ends the session
- Unknown commands return an error

10.

A production implementation would need

- TLS and certificate validation
- Strong authentication and authorization
- Explicit message framing and maximum frame sizes
- Username length and character validation
- Socket read/write timeouts
- Connection, message and rate limits
- Carefully managed async tasks
- Graceful handling of disconnects
- Logging with sensitive data removed and log injection prevention
- Input escaping when messages are displayed or stored
- Resource cleanup using context managers
- Replay and duplicate login protections where relevant
- Firewalling, network access controls, monitoring and alerting
- Secure secret and certificate storage
- Tests for malformed packets, oversized inputs, concurrency, disconnects and denial of service behavior

Part 11 Socketio Implementation, the code is in the Github

```
PS C:\Users\rkory\OneDrive\Desktop\FSC8561-Labs-Ryan\Lab 1> python server.py
===== Running on http://127.0.0.1:8080 =====
(Press CTRL+C to quit)
Connection established: 3a5RoCRyL1zHXCxkAAAB
Socket ID: 3a5RoCRyL1zHXCxkAAAB
Data: {'data': 'HELLO|Ryan'}
Username: Ryan
Socket ID: 3a5RoCRyL1zHXCxkAAAB
Data: {'data': 'hello'}
Ryan: hello
Socket ID: 3a5RoCRyL1zHXCxkAAAB
Data: {'data': 'goodbye'}
Ryan: goodbye
Disconnected: 3a5RoCRyL1zHXCxkAAAB
```

```
PS C:\Users\rkory\OneDrive\Desktop\FSC8561-Labs-Ryan\Lab 1> python .\client.py
connection established
Enter your username: Ryan
Enter message or type EXIT to leave: Server: Hello Ryan
hello
Enter message or type EXIT to leave: Server: Ryan: hello
goodbye
Enter message or type EXIT to leave: Server: Ryan: goodbye
exit
disconnected from server
```

Discussion:

After modifying my socketio code from last class, the server now maintains a users dictionary containing state for each Socket.IO connection instead of treating every message as an independent message.

Socket.IO handles the connection/event layer for you now, whereas the textbook's raw TCP implementation explicitly performs `socket()`, `bind()`, `listen()`, `accept()`, `recv()`, `send()`, and `connect()`.

Challenge

For this challenge, I demonstrated it using 2 clients

```
PS C:\Users\rkory\OneDrive\Desktop\FSC8561-Labs-Ryan\Lab 1> cd .\challenge\  
PS C:\Users\rkory\OneDrive\Desktop\FSC8561-Labs-Ryan\Lab 1\challenge> python .\server.py  
Server is waiting for connections...  
Connected by: ('127.0.0.1', 50535)  
Connected by: ('127.0.0.1', 59705)  
Received: HELLO|Ryan  
Username: Ryan  
Received: HELLO|Bob  
Username: Bob  
Received: MSG|Hello Everyone!  
Ryan says: Hello Everyone!  
Received: MSG|Hello!  
Bob says: Hello!  
Received: MSG|How are you?  
Ryan says: How are you?  
Received: MSG|Im Good!  
Bob says: Im Good!
```

Client 1

```
PS C:\Users\rkory\OneDrive\Desktop\FSC8561-Labs-Ryan\Lab 1> cd .\challenge\  
PS C:\Users\rkory\OneDrive\Desktop\FSC8561-Labs-Ryan\Lab 1\challenge> python .\client.py  
Enter your username: Ryan  
Server: OK|Hello Ryan  
Enter message or type EXIT to leave: Hello Everyone!  
Enter message or type EXIT to leave:  
OK|Message received from Ryan  
  
Bob: Hello!  
How are you?  
Enter message or type EXIT to leave:  
OK|Message received from Ryan  
  
Bob: Im Good!
```

Client 2

```
PS C:\Users\rkory\OneDrive\Desktop\FSC8561-Labs-Ryan\Lab 1\challenge> python .\client.py  
Enter your username: Bob  
Server: OK|Hello Bob  
Enter message or type EXIT to leave:  
Ryan: Hello Everyone!  
Hello!  
Enter message or type EXIT to leave:  
OK|Message received from Bob  
  
Ryan: How are you?  
Im Good!  
Enter message or type EXIT to leave:  
OK|Message received from Bob
```

Then when client 2 exits, the server stays on and client 1 is still active

```
PS C:\Users\rkory\OneDrive\Desktop\FSC8561-Labs-Ryan\Lab 1> cd .\challenge\  
PS C:\Users\rkory\OneDrive\Desktop\FSC8561-Labs-Ryan\Lab 1\challenge> python .\server.py  
Server is waiting for connections...  
Connected by: ('127.0.0.1', 50535)  
Connected by: ('127.0.0.1', 59705)  
Received: HELLO|Ryan  
Username: Ryan  
Received: HELLO|Bob  
Username: Bob  
Received: MSG|Hello Everyone!  
Ryan says: Hello Everyone!  
Received: MSG|Hello!  
Bob says: Hello!  
Received: MSG|How are you?  
Ryan says: How are you?  
Received: MSG|Im Good!  
Bob says: Im Good!  
Received: EXIT|  
Client connection closed: ('127.0.0.1', 59705)
```

```
PS C:\Users\rkory\OneDrive\Desktop\FSC8561-Labs-Ryan\Lab 1> cd .\challenge\  
PS C:\Users\rkory\OneDrive\Desktop\FSC8561-Labs-Ryan\Lab 1\challenge> python .\client.py  
Enter your username: Ryan  
Server: OK|Hello Ryan  
Enter message or type EXIT to leave: Hello Everyone!  
Enter message or type EXIT to leave:  
OK|Message received from Ryan  
  
Bob: Hello!  
How are you?  
Enter message or type EXIT to leave:  
OK|Message received from Ryan  
  
Bob: Im Good!
```

```
PS C:\Users\rkory\OneDrive\Desktop\FSC8561-Labs-Ryan\Lab 1\challenge> python .\client.py  
Enter your username: Bob  
Server: OK|Hello Bob  
Enter message or type EXIT to leave:  
Ryan: Hello Everyone!  
Hello!  
Enter message or type EXIT to leave:  
OK|Message received from Bob  
  
Ryan: How are you?  
Im Good!  
Enter message or type EXIT to leave:  
OK|Message received from Bob  
exit  
  
OK|Goodbye  
Server:  
Disconnected
```

For the server file it used to be

```
client_socket, client_address = server_socket.accept()
```

And then it is modified to handle the clients
while True:

```
client_socket, client_address = server_socket.accept()

client_thread = threading.Thread(
    target=handle_client,
    args=(client_socket, client_address)
)

client_thread.start()
```

For the client it remains almost the same, except when the server receives a MSG, the server sends that message to the other clients, so this client must be able to receive messages that it did not directly request.

```
def receive_messages():

    while True:

        data = client_socket.recv(1024)

        if not data:
            break

        print("\n" + data.decode())
```

Challenge Reflection

1.
The original server calls `accept()` once and then enters a loop communicating with that client. It does not return to `accept()` to accept another client until that connection finishes
2.
Threading allows each connected client to be handled independently. The server can communicate with one client while another client's thread handles a different connection
3.
Only that client's thread and socket are closed. The server continues running, and the other connected clients can continue communicating.
4.
Unauthorized users connecting, sending malicious input, impersonating users, and overwhelming the server with too many connections or messages

5.

In our implementation right now, the server would have 2 different connections associated with the same username. Messages would be ambiguous and both users could appear to have the same identity.

6.

The server should authenticate users rather than trusting a username supplied in HELLO|username. Ex: users could authenticate with credentials, and the server could associate the authenticated identity with the client's socket.

Security Analysis

Transitioning from a basic stateless program to a persistent, stateful application changes how the server interacts with clients, which introduces both new validation requirements and fresh security risks. In this implementation, the server uses a username state variable to enforce the application protocol. For example, if a user attempts to send a MSG packet before sending a HELLO packet, the server catches this and returns an ERROR|HELLO required first response.

The server also relies on input validation to prevent invalid data from causing unexpected crashes. It checks that incoming strings contain the required | delimiter. If the delimiter is missing, it rejects the input with an ERROR|Invalid command format message. It also checks payload boundaries, returning ERROR|Message cannot be empty for blank messages and ERROR|Message too long for inputs exceeding 200 characters. These checks help stop simple buffer overflow attempts and prevent empty data from being processed.

Despite these controls, many vulnerabilities remain in the application design. Because the connection relies on plain-text TCP without TLS encryption, all transmitted data, including usernames and message contents, can be read or altered using packet sniffing tools. Additionally, relying on `recv(1024)` is unsafe for protocol framing. TCP is a continuous stream of bytes rather than a set of discrete messages. This means a single `recv()` call might split a longer message into multiple fragments or combine multiple small messages together, which can break input length checks or lead to command parsing errors.

Finally, the application lacks essential security controls like authentication, rate limiting, and connection timeouts. Usernames are accepted without passwords or verification, making identity spoofing trivial. Furthermore, because the server is single-threaded and handles only one socket at a time, a single user can easily cause a Denial of Service (DoS) by keeping the port open and ignoring input prompts. Real-world deployment would require TLS, multi-threading, proper message framing headers, and credential checks to keep the service secure.